

1.整体设计思路

1.1 架构设计

状态管理：使用 Zustand 进行全局状态管理，所有画布状态、工具设置、视图状态都集中在 `usewhiteboardStore` 中

组件化设计：采用组件化架构，主要组件包括：

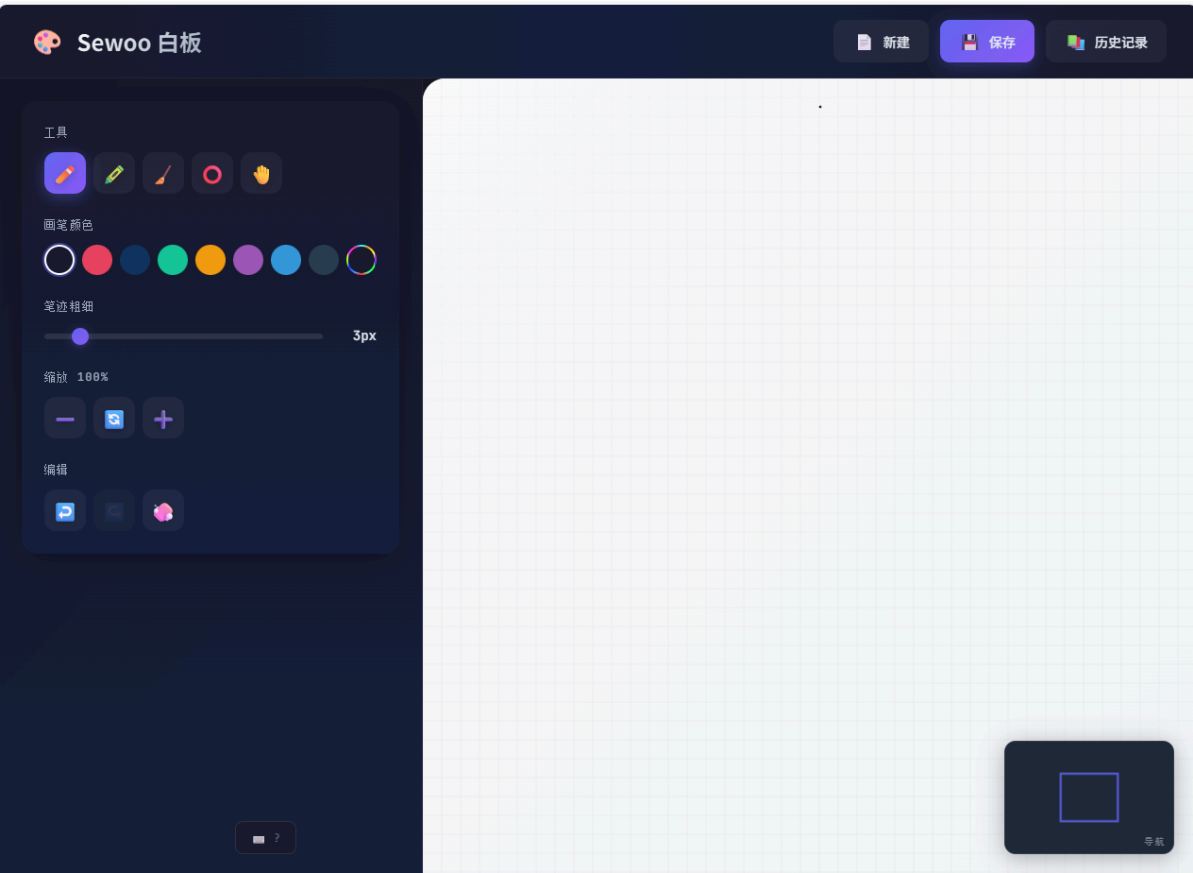
`Canvas`：核心画布组件，负责绘制和交互

`Toolbar`：工具栏组件，提供工具选择和设置

`Header`：顶部栏组件

`HistorySidebar`：历史记录侧边栏

`MiniMap`：小地图组件



总体图

1.2 笔记数据定义

笔记在代码中被定义为 Stroke（笔触），其数据结构如下：

```
interface Stroke {  
  id: string;          // 唯一标识符（使用 UUID）  
  points: Point[];     // 笔触的所有点坐标  
  color: string;        // 颜色值  
  width: number;        // 笔触宽度  
  type: 'pen' | 'highlighter'; // 笔触类型  
}  
  
interface Point {  
  x: number;           // X 坐标  
  y: number;           // Y 坐标
```

```
pressure?: number;    // 压力值（可选，用于压感笔）
}
```

1.3 画布状态管理

画布的整体状态通过 `CanvasState` 管理：

```
interface CanvasState {
  strokes: Stroke[];          // 所有笔触的数组
  selectedStrokeIds: string[]; // 选中笔触的 ID 数组
  selectionBox: SelectionBox | null; // 选择框（用于显示选中区域）
}
```

设计要点：

使用 ID 数组而非对象数组来存储选中状态，减少内存占用

通过 `selectionBox` 统一管理选中区域的显示，避免重复计算边界框

所有笔触存储在 `strokes` 数组中，通过 ID 进行引用和查找

1.4 视图变换系统

应用支持平移和缩放功能，通过 `ViewState` 管理：

```
interface ViewState {
  scale: number; // 缩放比例
  offsetX: number; // X 轴偏移
  offsetY: number; // Y 轴偏移
}
```

1.5 撤销/重做机制

使用栈结构实现撤销/重做：

`undoStack`：撤销栈，保存历史状态（最多 50 个）

`redoStack`：重做栈，保存被撤销的状态

每次修改画布时，都会将当前状态保存到撤销栈中。

2. 笔迹生成

2.1 笔迹生成的生命周期

笔迹生成分为三个阶段：开始 → 绘制中 → 结束

1：开始绘制（startStroke）

当用户按下鼠标时，触发 `handleMouseDown` 事件：

将笔触存储在 `currentStroke` 中（临时状态，尚未保存到画布）

2：绘制中（addPointToStroke）

鼠标移动时，持续添加点到当前笔触：每次鼠标移动都会添加一个新点到 `points` 数组，使用展开运算符创建新数组，保持不可变性，坐标已转换为画布坐标系

3: 结束绘制 (endStroke) 鼠标抬起时, 完成笔触并保存到画布:

1. 将 `currentStroke` 添加到 `canvasState.strokes` 数组
2. 清空 `currentStroke` (准备下一次绘制)
3. 保存当前状态到撤销栈
4. 清空重做栈 (新操作后无法重做之前被撤销的操作)

2.2 笔迹的实时渲染

在绘制过程中, `currentStroke` 会被实时渲染到画布上: 这样用户可以看到实时的绘制效果, 而不是等到鼠标抬起才显示。

2.3 平滑曲线算法 (Catmull-Rom 样条)

为了生成平滑的笔迹, 使用 **Catmull-Rom 样条曲线算法:

Catmull-Rom 样条是一种插值样条, 通过四个控制点生成平滑曲线。对于笔触中的每两个相邻点, 使用前后各一个点作为控制点。

2.4 高亮笔特殊处理

高亮笔使用特殊的混合模式: `multiply` (正片叠底), 使高亮笔颜色与底层内容混合

2.5 坐标系转换

绘制过程中涉及多次坐标转换, 确保无论画布如何缩放和平移, 笔迹都能正确保存和显示。

3.橡皮擦实现原理

3.1 核心思路

橡皮擦的实现采用 切割算法: 当橡皮擦划过笔触时, 不是直接删除整个笔触, 而是将被擦除的部分切割掉, 保留未被擦除的部分, 并生成新的笔触段。

3.2 实现流程

步骤 1: 坐标转换

将屏幕坐标转换为画布坐标, 考虑当前的缩放和平移。

步骤 2: 遍历所有笔触并切割

对每个笔触调用 `splitStrokeAtPoint` 进行切割, 如果笔触未被影响, 则保留原样; 如果被切割, 则用新的笔触段替换。

步骤 3: 切割算法核心逻辑

*算法说明**:

1. 遍历笔触的每个点, 计算该点到橡皮擦中心的距离

2. 如果点距离橡皮擦中心 大于半径, 则保留该点, 加入当前段
如果点距离橡皮擦中心 小于等于半径, 则:
如果当前段有足够的点 (>1), 保存为一个新段
清空当前段, 开始新的段
 3. 生成新笔触: 每个保留的段都会生成一个新的笔触对象, 使用新的 UUID
- ### 3.3 优化点

4. 选择框功能实现

4.1 选择框数据结构

选择框通过 `SelectionBox` 接口定义, 用于表示选中区域的边界框:

```
interface SelectionBox {  
  x: number;    // 选择框左上角 X 坐标 (画布坐标系)  
  y: number;    // 选择框左上角 Y 坐标 (画布坐标系)  
  width: number; // 选择框宽度  
  height: number; // 选择框高度  
}
```

选择框存储在 `CanvasState` 中, 与 `selectedStrokeIds` 配合使用, 共同管理选中状态。

4.2 选择框的创建

选择框通过套索工具 (Lasso Tool) 创建, 整个选中过程分为三个阶段:

阶段 1: 开始套索选择 (startLasso)

当用户按下鼠标开始绘制套索路径时:

- 初始化 `lassoPoints` 数组, 存储第一个点 (屏幕坐标系)
- 设置绘制状态, 准备收集套索路径的点

阶段 2: 绘制套索路径 (addLassoPoint)

鼠标移动时持续添加点到套索路径:

- 每次鼠标移动都会调用 `addLassoPoint`, 将当前点添加到 `lassoPoints` 数组
- 套索路径以屏幕坐标系存储, 实时显示在画布上
- 路径使用虚线样式绘制, 颜色为 `#6366f1`

阶段 3: 完成选择并检测笔触 (endLasso)

当用户释放鼠标完成套索绘制时, 首先进行有效性检查:

- 如果套索路径的点数少于 3 个, 无法形成有效的多边形, 直接清空并返回
- 只有点数 ≥ 3 时, 才执行后续的检测和选择流程

然后执行以下步骤:

1. **坐标转换:** 将套索路径从屏幕坐标系转换为画布坐标系

```
const canvasLassoPoints = lassoPoints.map(p => ({  
  x: (p.x - offsetX) / scale,  
  y: (p.y - offsetY) / scale,  
}));
```

这一步确保检测算法使用正确的坐标系统

2. **检测笔触是否在套索内**：遍历所有笔触，使用 `isStrokeInsideLasso` 函数判断

- 对于每个笔触，检查其所有点是否至少有一个在套索多边形内
- 如果笔触的任何一个点在多边形内，则认为该笔触被选中
- 将选中笔触的 ID 添加到 `selectedIds` 数组

3. **计算边界框**：对于所有选中的笔触，计算它们的整体边界框

- 遍历所有选中笔触的所有点
- 找出最小和最大的 X、Y 坐标
- 创建选择框，并添加 10 像素的边距（上下左右各 10 像素）

4. **更新状态**：将选中结果保存到状态中

- `selectedStrokeIds`：保存所有选中笔触的 ID 数组
- `selectionBox`：保存计算出的边界框
- 清空 `lassoPoints`，准备下一次选择

4.3 笔触检测算法（isStrokeInsideLasso）

检测笔触是否在套索内的核心逻辑：

```
function isStrokeInsideLasso(stroke: Stroke, lassoPoints: Point[]): boolean {  
  // 检查笔触的任何一个点是否都在多边形内  
  for (const point of stroke.points) {  
    if (isPointInPolygon(point, lassoPoints)) {  
      return true; // 只要有一个点都在多边形内，就认为笔触被选中  
    }  
  }  
  return false;  
}
```

设计要点：

- 只要笔触的任何一个点在套索内，整个笔触就被选中
- 只要笔触的一部分被套索包含，就应该被选中
- 对于跨越套索边界的笔触，也能正确识别

4.4 点内多边形检测算法（射线投射算法）

判断点是否都在多边形内使用经典的**射线投射算法**（Ray Casting Algorithm）：

```
function isPointInPolygon(point: Point, polygon: Point[]): boolean {  
  let inside = false;  
  const n = polygon.length;  
  
  // 遍历多边形的每条边  
  for (let i = 0, j = n - 1; i < n; j = i++) {  
    const xi = polygon[i].x, yi = polygon[i].y;  
    const xj = polygon[j].x, yj = polygon[j].y;  
  
    // 检查点是否在边的水平范围内，且射线与边相交  
    if (((yi > point.y) !== (yj > point.y)) &&  
        (point.x < (xj - xi) * (point.y - yi) / (yj - yi) + xi)) {  
      inside = !inside;  
    }  
  }  
  return inside;  
}
```

```
        inside = !inside; // 每相交一次，状态翻转
    }
}

return inside;
}
```

算法原理：

1. 从待检测点向右（或任意方向）发射一条水平射线
2. 统计射线与多边形边的相交次数
3. 如果相交次数为奇数，则点在多边形内；如果为偶数，则点在多边形外
4. 算法时间复杂度： $O(n)$ ， n 为多边形顶点数

5.性能优化

5.1 优化策略概览

当用户全选多个笔记时，应用采用了以下性能优化策略：

5.2 详细优化实现

优化 1：使用 ID 数组存储选中状态

优化 2：选择框 (SelectionBox) 缓存

在选择完成后，一次性计算所有选中笔触的边界框，并缓存到 `selectionBox` 中：

优化 3：批量移动优化

只更新选中笔触：使用 `map` 遍历所有笔触，但只修改选中笔触的坐标

同步更新选择框：移动时直接更新选择框位置，避免重新计算

坐标缩放处理：考虑视图缩放，确保移动距离正确

优化4：使用 Set 优化查找（准备优化）

// 当前方式 ($O(n)$)

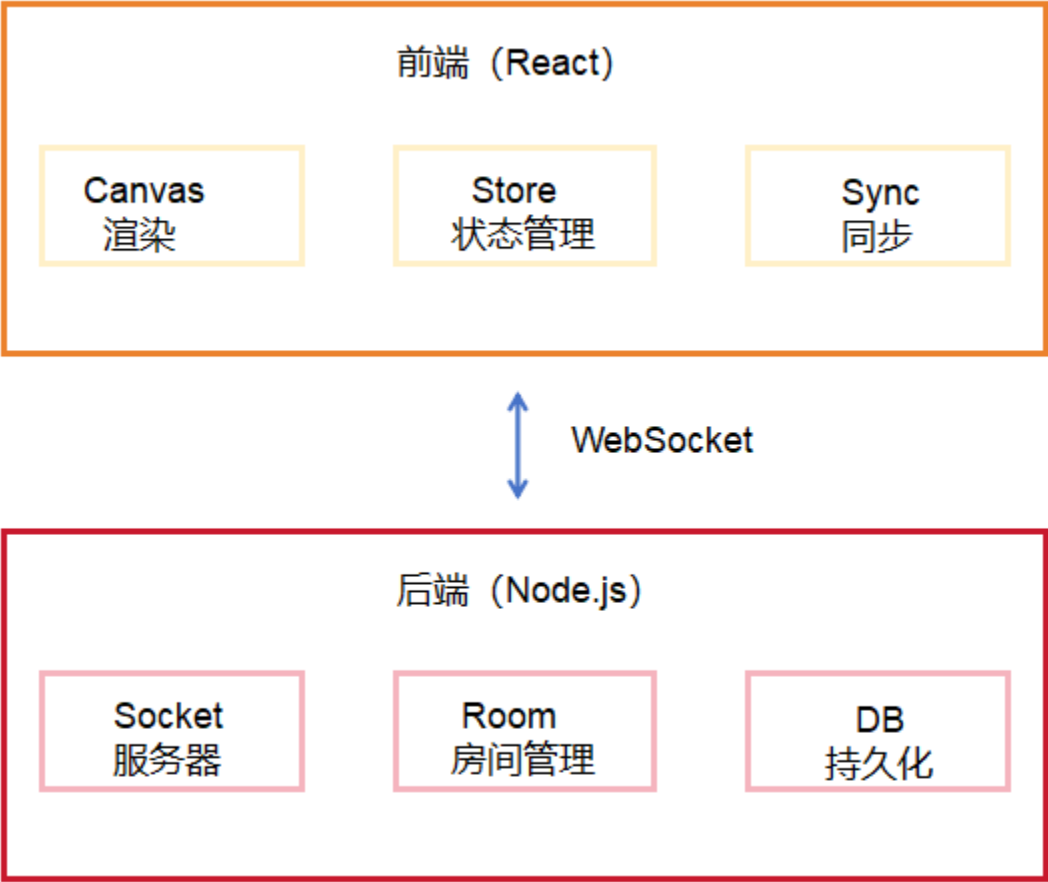
`selectedStrokelds.includes(stroke.id)`

// 优化后 ($O(1)$)

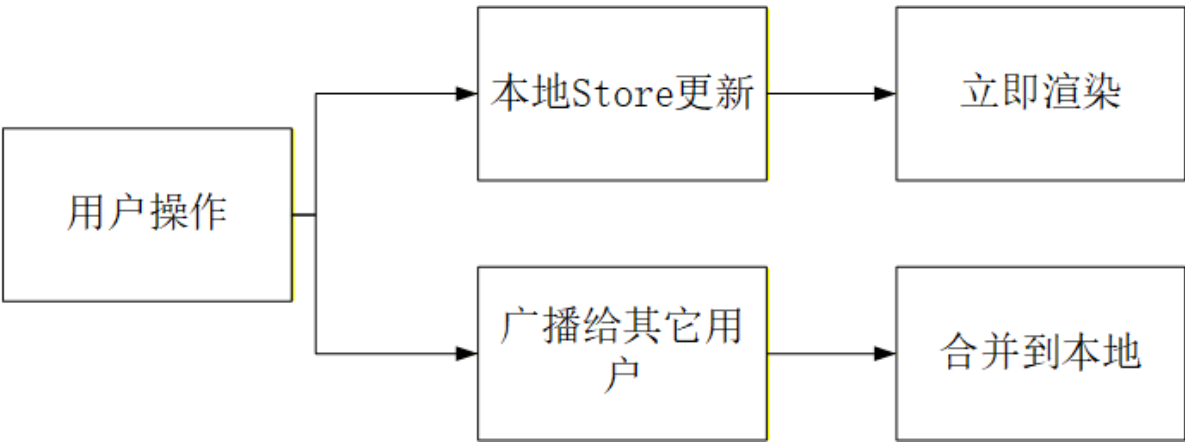
`selectedStrokeldsSet.has(stroke.id)`

6.互动白板

6.1整体框架



6.2数据流操作



6.3同步策略

实时操作 + 定期快照校验

操作级同步用于实时性

快照用于断线重连和一致性校验

6.4冲突解决策略

时间戳优先